NaN, the not-a-number number that isn’t NaN

Mat “Wilto” Marquis, *23 October 2025* *Topic:* JavaScript

When `NaN` is included in a arithmetic expression, the result will always be `NaN` — that tracks, right? Anything math’d against the very concept of “not a number” can’t result in a number:

JS COPY TO CLIPBOARD

```
2 + NaN;
// result: NaN

NaN - 50;
// result: NaN

NaN / 0;
// result: NaN
```

That means once any *part* of a calculation includes or results in `NaN`, the whole thing will result in `NaN`. As soon as `NaN` is in play, we can’t possibly end up with a number:

JS COPY TO CLIPBOARD

```
( 2 + 2 ) * 10 / ( "Ten" * 4 ) + 9;
// result: NaN
```

Likewise, any comparison that uses `NaN` as one of the operands will evaluate to `false`, which certainly tracks in the same way — no value can be greater than, less than, or equal to what is effectively a placeholder for the concept of being a non-number result:

JS COPY TO CLIPBOARD

```
50 > NaN;
// result: false
```

It follows that any individual value will be unequal to `NaN`, as those values either *are* numbers — thus not `NaN` — or aren't evaluated *as* numbers, and thus not `NaN`.

JS COPY TO CLIPBOARD

```
100 !== NaN;
// result: true

"String" !== NaN;
// result: true
```

Now, here's where it gets weird: that inequality extends to `NaN` itself. The way `true` represents the very essence of trueness, `NaN` represents a non-specific non-number result. `NaN` is the only value in the whole of JavaScript that isn't equal to itself.

JS COPY TO CLIPBOARD

```
NaN == NaN;
// result: false

NaN === NaN;
// result: false

NaN !== NaN;
// result: true
```

Now, the cheap explanation is "well, that's because `NaN` *is* a number."

JS COPY TO CLIPBOARD

```
typeof NaN;
// result: number
```

By definition a number can't be equal to the concept of not-a-number, sure, but `NaN !== NaN` goes much deeper than that, and well beyond JavaScript itself. Across the whole of computer programming, the concept of `NaN` is meant to represent a breakdown of calculation — the end result of any mathematical equation that comes to involve a `NaN` value, no matter how simple or complex it may be, must end in `NaN`. `NaN` is, in effect, an error state.

> “
>
> An operation that propagates a NaN operand to its result and has a single NaN as an input should produce a NaN with the payload of the input NaN if representable in the destination format.
>
> — IEEE Std 754-2019, IEEE Standard for Floating-Point Arithmetic

We're operating strictly in the realm of calculations, here. In order to function like an error in a calculation without *itself* causing wildly unpredictable results in that calculation, `NaN` has to behave like a number. That's why `NaN` is a number.

That's also the reason `NaN !== NaN`. If `NaN` behaved like a number *and* had a value equal to itself, well, you could accidentally do math with it: `NaN / NaN` would result in `1`, and that would mean that a calculation containing a `NaN` result could ultimately result in an incorrect number rather than an easily-spotted “hey, something went wrong in here” `NaN` flag.

Now, as you might imagine, this makes it tricky to determine whether an expression has evaluated to `NaN`. Say I want to multiply the value assigned to a given identifier by ten *only if that value is a number* — I might write the following, thinking “well, if it *isn't* `NaN`, it must be a number, so do math to it; otherwise, do something else:“

Try it out

```
let theValue = "String";

if ( theValue != NaN ) {
  console.log( theValue * 10 );
} else {
  console.log( "This isn't a number." );
}
```

RUN

No dice, because the simple expression `"String"` doesn't evaluate to the concept of a non-number value in a the context of a calculation, it's a string. It evaluates to a string. We might then think, "okay, fine, we'll be explicit: do the math, and the result of *that* is equal to `NaN`, do this, else do the math:"

Try it out

```
let theValue = "String";

if ( theValue * 10 === NaN ) {
  console.log( "This isn't a number." );
} else {
  console.log( theValue * 10 );
}
```

RUN

Still no good. `theValue * 10` does evaluate to `NaN`, but `NaN` isn't equal to `NaN`.

Instead, we have a couple of options: there is, of course, using good old-fashioned `typeof` to see if we're working with a number:

Try it out

```
let theValue = "String";

if ( typeof theValue === "number" ) {
  console.log( theValue * 10 );
} else {
  console.log( "This isn't a number." );
}
```

RUN

Reliable, though comparing strings in order to verify that something is a number has always felt a little clunky to me. Luckily, we can also make use of the global method `isNaN` — which has existed since the very first ECMAScript specification in 1997 — and the `Number.isNaN` method introduced in ES6.

JS | COPY TO CLIPBOARD

```
isNaN( "Two" * 2 );
// result: true

isNaN( 20 );
// result: false

Number.isNaN( "Two" * 2 );
// result: true
```

There is — I say with a depth of sigh that can only come from experience — a big difference between `isNaN()` and `Number.isNaN()`. If you've made it this far, though, you're through the worst `NaN` has to offer. We're in the home stretch here.

You can think of the global `isNaN` method as checking to see whether something *is not a number*, or perhaps more accurately, "if I tried to make you into a number, would that work, or would you end up being `NaN`?" An expression supplied to `isNaN()` will coerce the resulting value to a number, and if the result of that coercion is `NaN`, the method returns `true`:

JS | COPY TO CLIPBOARD

```
isNaN( "Two" * 2 );
// result: true
```

```
isNaN( 20 );
// result: false

isNaN( "20" );
// result: false

isNaN( "A string" );
// result: true
```

You can think of the `Number.isNaN` method as checking to see whether something *is* the value `NaN`, just like it says on the tin. It doesn't perform any coercion — it just checks to see whether `NaN` is the explicit result of the expression you've given it:

JS | COPY TO CLIPBOARD

```
Number.isNaN( "Two" * 2 );
// result: true

Number.isNaN( 20 );
// result: false

Number.isNaN( "20" );
// result: false

Number.isNaN( "A string" );
// result: false
```

`"Two" * 2` results in `NaN`, no two ways about it; both methods return `true`.

The number `20`, being a number and all, is not `NaN`, nor does it evaluate to `NaN`; both methods return `false`.

The string `"20"` *can be* coerced to a number value, so `isNaN` returns `false`. `Number.isNaN` doesn't try to coerce `"20"` to a number, but that value still isn't `NaN`, it's a string. `false` there too.

When `isNaN` tries to coerce `"A string"` to a number value, *that* results in `NaN`, so `isNaN` returns `true`. But once again: a string doesn't evaluate to `NaN` in and of itself. `Number.isNaN( "A string" )` returns `false`.

So for purposes of our snippet, **the global `isNaN` is the tool for the job**. If this *can* be evaluated to a number, it *will* be evaluated to a number when we attempt to multiply it by ten.

Try it out

```
let theValue = "Ten";

if ( isNaN( theValue ) ) {
  console.log( "This isn't a number." );
} else {
  console.log( theValue * 10 );
}
```

RUN

`"Ten"` can’t be coerced to a number, so this works as expected; just like the majority of my high school career, no math is attempted whatsoever. The string `"10"` *can* be coerced to the number value `10` though, and that works too:

Try it out

```
let theValue = "10";

if ( isNaN( theValue ) ) {
  console.log( "This isn't a number." );
} else {
  console.log( theValue * 10 );
}
```

RUN

If we ultimately wanted to check against an *explicit* `NaN` value without performing any coercion whatsoever — a use case arguably more in-line with the IEEE intent of `NaN` as a sort of error state — we’d want to use `Number.isNaN` instead:

Try it out

```
let theResult = "10" * 10;

if ( Number.isNaN( theResult ) ) {
  console.log( "The calculation hasn't resulted in a number." );
} else {
  console.log( theResult );
}
```

▷ RUN

So there you have it: `NaN`, the number that means “not a number” is a number, but it isn’t `NaN` . And they say JavaScript is confusing.

Enjoyed this article? *You can support us by leaving a tip via Open Collective*

Mat “Wilto” Marquis

AUTHOR

Independent front-end developer, designer, author of Javascript For Web Designers, JavaScript for Everyone, and hobby collector.

CHECK OUT MAT’S JAVASCRIPT COURSE ↗

» More about Mat “Wilto” Marquis